


```
from torch.utils.data import Dataset, DataLoader, random_split
import segmentation_models_pytorch as smp
import torch.optim as optim

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"🔥 Running on: {device}")
```

🔥 Running on: cuda

▼ Part 4: Augmentation Pipeline

v3 augmentations (vs v2 which only had h/v flips):

Augmentation	Why it helps for SAR flood mapping
HorizontalFlip / VerticalFlip	Basic orientation invariance
Rotate ($\pm 30^\circ$)	Flood shapes are irregular — rotation teaches texture, not just orientation
RandomScale (0.8–1.2)	Floods appear at different spatial scales
ElasticTransform	Deforms flood boundaries realistically — SAR flood edges are never perfectly sharp
GaussNoise	Simulates SAR speckle noise (SAR has multiplicative noise, not additive Gaussian, but this is a good approximation)
RandomBrightnessContrast	Simulates variation in incidence angle and scene moisture across different acquisitions

All transforms applied identically to both image and mask (albumentations handles this automatically via `additional_targets`).

```
# Augmentation pipeline – applied to TRAINING set only
# RandomScale changes spatial dimensions, so we always crop/pad back to
# a fixed size (512x512) at the end so batches can be stacked.
TARGET_SIZE = 512

train_transform = A.Compose([
    A.HorizontalFlip(p=0.5),
    A.VerticalFlip(p=0.5),
    A.Rotate(limit=30, p=0.5, border_mode=0),
    A.RandomScale(scale_limit=0.2, p=0.4),          # zoom 80-120%
    # After scaling, images may be different sizes – pad then crop to TARGET_SIZE
    A.PadIfNeeded(min_height=TARGET_SIZE, min_width=TARGET_SIZE,
                  border_mode=0, value=0, mask_value=0),
    A.RandomCrop(height=TARGET_SIZE, width=TARGET_SIZE),
    A.ElasticTransform(
        alpha=1, sigma=10, p=0.3
    ),
    A.GaussNoise(std_range=(0.01, 0.05), p=0.4),  # speckle approximation
    A.RandomBrightnessContrast(brightness_limit=0.1, contrast_limit=0.1, p=0.3)
], additional_targets={"mask": "mask"}, is_check_shapes=False)

print(f"✅ Augmentation pipeline defined. Output size: {TARGET_SIZE}x{TARGET_SIZE}")
```

✓ Augmentation pipeline defined. Output size: 512x512
 /tmp/ipykernel_563/791677783.py:12: UserWarning: Argument(s) 'value, mask_value'
 A.PadIfNeeded(min_height=TARGET_SIZE, min_width=TARGET_SIZE,

✓ Part 5: Dataset with 72 / 18 / 10 Split

```
class Sen1FloodsDataset(Dataset):
    """Base dataset – loads and preprocesses, NO augmentation."""
    def __init__(self, data_dir):
        self.image_paths = sorted(glob.glob(
            os.path.join(data_dir, "**", "*_S1Hand.tif"), recursive=True
        ))
        self.mask_paths = [
            p.replace("S1Hand", "LabelHand") for p in self.image_paths
        ]
        print(f"✓ Found {len(self.image_paths)} image/mask pairs.")

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        with rasterio.open(self.image_paths[idx]) as src:
            image = src.read().astype(np.float32)  # (2, H, W)
            image = np.nan_to_num(image)
            image = np.clip(image, -30, 0)
            image = (image + 30) / 30.0

        with rasterio.open(self.mask_paths[idx]) as src:
            mask = src.read(1).astype(np.float32)  # (H, W)
            mask = np.where(mask == 1, 1.0, 0.0)

        image_tensor = torch.tensor(image, dtype=torch.float32)
        mask_tensor = torch.tensor(mask, dtype=torch.float32).unsqueeze(0)
        return image_tensor, mask_tensor

class AugmentedSubset(Dataset):
    """Wraps a Subset and applies albumentations transforms on the fly.
    albumentations expects HWC for images, HW for masks.
    """
    def __init__(self, subset, transform=None):
        self.subset = subset
        self.transform = transform

    def __len__(self):
        return len(self.subset)

    def __getitem__(self, idx):
```

```

        image, mask = self.subset[idx]          # tensors: (2,H,W), (1,H,W)
        if self.transform is not None:
            # albumentations needs numpy HWC / HW
            img_np = image.numpy().transpose(1, 2, 0)  # (H,W,2)
            mask_np = mask.numpy().squeeze(0)          # (H,W)
            aug = self.transform(image=img_np, mask=mask_np)
            image = torch.tensor(
                aug["image"].transpose(2, 0, 1), dtype=torch.float32
            ) # back to (2,H,W)
            mask = torch.tensor(
                aug["mask"], dtype=torch.float32
            ).unsqueeze(0) # back to (1,H,W)
        return image, mask

# — Split —————
full_dataset = Sen1FloodsDataset("/content/sen1floods11/HandLabeled/")

total      = len(full_dataset)
test_size  = int(0.10 * total)
val_size   = int(0.18 * total)
train_size = total - test_size - val_size

train_base, val_dataset, test_dataset = random_split(
    full_dataset, [train_size, val_size, test_size],
    generator=torch.Generator().manual_seed(42)
)

train_dataset = AugmentedSubset(train_base, transform=train_transform)
val_aug       = AugmentedSubset(val_dataset, transform=None) # no aug on val
test_aug      = AugmentedSubset(test_dataset, transform=None) # no aug on tes

train_loader = DataLoader(train_dataset, batch_size=4, shuffle=True, num_worke
val_loader   = DataLoader(val_aug, batch_size=4, shuffle=False, num_worke
test_loader  = DataLoader(test_aug, batch_size=4, shuffle=False, num_worke

print(f"Train: {len(train_dataset)} | Val: {len(val_aug)} | Test: {len(test_aug)

✓ Found 446 image/mask pairs.
Train: 322 | Val: 80 | Test: 44

```

✓ Part 6: Model — EfficientNet-B3 backbone

v3 backbone change: ResNet34 → EfficientNet-B3

Backbone	Params	ImageNet top-1	Notes
ResNet34	21M	73.3%	v1 + v2 baseline
ResNet50	25M	76.1%	Better but heavy
EfficientNet-B3	12M	81.6%	Fewer params, significantly better feature extraction

EfficientNet was designed with compound scaling (width + depth + resolution together) which gives it much richer multi-scale feature maps — critical for SAR segmentation where flood boundaries appear at many scales simultaneously. Same U-Net decoder, just swap `encoder_name`.

```
model = smp.Unet(
    encoder_name="efficientnet-b3",      # upgraded from resnet34
    encoder_weights=None,               # training from scratch on SAR data
    in_channels=2,                      # VV + VH
    classes=1,
).to(device)

total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"✅ Model: U-Net + EfficientNet-B3 | Trainable params: {total_params}")
```

✅ Model: U-Net + EfficientNet-B3 | Trainable params: 13,158,673

✓ Part 7: Loss Function — Dice + Focal Loss

v3 loss change: SoftBCE → Focal Loss

`pos_weight=10` in BCE is a blunt instrument — it multiplies the loss for ALL flood pixels equally regardless of how easy or hard they are to predict.

Focal Loss solves this properly:

- Easy pixels (model already confident) → loss weight close to 0
- Hard pixels (model uncertain, near boundary) → loss weight stays high
- Automatically focuses training on the pixels that matter most
- `gamma=2` is the standard setting from the original paper (Lin et al. 2017)
- `alpha=0.75` upweights the flood class (minority class in SAR data)

Combined: `DiceLoss + FocalLoss` = overlap maximisation + hard-pixel focus

```
dice_loss = smp.losses.DiceLoss(mode="binary")
focal_loss = smp.losses.FocalLoss(
    mode="binary",
    alpha=0.75,      # upweight flood class (minority class)
    gamma=2.0,       # focus on hard/uncertain pixels
)

def criterion(outputs, masks):
    return dice_loss(outputs, masks) + focal_loss(outputs, masks)

# Lower starting LR (0.00005 vs 0.0001 in v2) — reduces val metric noise
```

```
optimizer = optim.Adam(model.parameters(), lr=0.00005)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode="min", patience=4, factor=0.5
)

print("✅ Loss: DiceLoss + FocalLoss(alpha=0.75, gamma=2)")
print("✅ Optimizer: Adam lr=0.00005")
```

```
✅ Loss: DiceLoss + FocalLoss(alpha=0.75, gamma=2)
✅ Optimizer: Adam lr=0.00005
```

▼ Part 8: Metric Helpers

```
def iou_score(pred, target, threshold=0.5):
    pred = (torch.sigmoid(pred) > threshold).float()
    intersection = (pred * target).sum()
    union = pred.sum() + target.sum() - intersection
    return (intersection + 1e-6) / (union + 1e-6)

def dice_score(pred, target, threshold=0.5):
    pred = (torch.sigmoid(pred) > threshold).float()
    intersection = (pred * target).sum()
    return (2 * intersection + 1e-6) / (pred.sum() + target.sum() + 1e-6)

print("✅ Metrics ready.")
```

```
✅ Metrics ready.
```

▼ Part 9: Training Loop — 50 Epochs

v3 trains for 50 epochs (was 30). The v2 training loss was still declining at epoch 30, meaning the model had not converged. 50 epochs with the lower LR (0.00005) should reach a stable minimum.

```
def train_model(model, train_loader, val_loader, epochs=50):
    print("🚀 Starting v3 training...")
    best_val_loss = float("inf")
    history = []

    for epoch in range(epochs):

        # — Train —————
        model.train()
        t_loss = t_iou = t_dice = 0.0
        for images, masks in train_loader:
```

```

        images, masks = images.to(device), masks.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, masks)
        loss.backward()
        optimizer.step()
        t_loss += loss.item()
        t_iou += iou_score(outputs, masks).item()
        t_dice += dice_score(outputs, masks).item()
    n_tr = len(train_loader)

    # — Validate —————
    model.eval()
    v_loss = v_iou = v_dice = 0.0
    with torch.no_grad():
        for images, masks in val_loader:
            images, masks = images.to(device), masks.to(device)
            outputs = model(images)
            v_loss += criterion(outputs, masks).item()
            v_iou += iou_score(outputs, masks).item()
            v_dice += dice_score(outputs, masks).item()
    n_val = len(val_loader)
    avg_val_loss = v_loss / n_val
    scheduler.step(avg_val_loss)

    tag = ""
    if avg_val_loss < best_val_loss:
        best_val_loss = avg_val_loss
        torch.save(model.state_dict(), "/content/flood_unet_resnet34_best.p
        tag = " 🌟 BEST SAVED"

    lr = optimizer.param_groups[0]["lr"]
    row = dict(
        epoch=epoch+1,
        train_loss=t_loss/n_tr, train_iou=t_iou/n_tr, train_dice=t_dice/n_t
        val_loss=avg_val_loss, val_iou=v_iou/n_val, val_dice=v_dice/n_val,
        lr=lr
    )
    history.append(row)

    print(
        f"Epoch {epoch+1:02d}/{epochs} | "
        f"Train Loss: {t_loss/n_tr:.4f} | "
        f"Train IoU: {t_iou/n_tr:.4f} | Train Dice: {t_dice/n_tr:.4f} | "
        f"Val Loss: {avg_val_loss:.4f} | "
        f"Val IoU: {v_iou/n_val:.4f} | Val Dice: {v_dice/n_val:.4f} | "
        f"LR: {lr:.6f}{tag}"
    )

    print(f"\n✅ Done. Best val loss: {best_val_loss:.4f}")
    return history

```

```
history = train_model(model, train_loader, val_loader, epochs=50)
```

 Starting v3 training...

Epoch 01/50	Train Loss: 0.8830	Train IoU: 0.1504	Train Dice: 0.2418	Val
Epoch 02/50	Train Loss: 0.7927	Train IoU: 0.2896	Train Dice: 0.4102	Val
Epoch 03/50	Train Loss: 0.7791	Train IoU: 0.2976	Train Dice: 0.4124	Val
Epoch 04/50	Train Loss: 0.7441	Train IoU: 0.3277	Train Dice: 0.4416	Val
Epoch 05/50	Train Loss: 0.7308	Train IoU: 0.3331	Train Dice: 0.4519	Val
Epoch 06/50	Train Loss: 0.6955	Train IoU: 0.3537	Train Dice: 0.4793	Val
Epoch 07/50	Train Loss: 0.6920	Train IoU: 0.3428	Train Dice: 0.4653	Val
Epoch 08/50	Train Loss: 0.6947	Train IoU: 0.3377	Train Dice: 0.4558	Val
Epoch 09/50	Train Loss: 0.6744	Train IoU: 0.3571	Train Dice: 0.4866	Val
Epoch 10/50	Train Loss: 0.6668	Train IoU: 0.3533	Train Dice: 0.4822	Val
Epoch 11/50	Train Loss: 0.6431	Train IoU: 0.3834	Train Dice: 0.5144	Val
Epoch 12/50	Train Loss: 0.6548	Train IoU: 0.3604	Train Dice: 0.4851	Val
Epoch 13/50	Train Loss: 0.6695	Train IoU: 0.3465	Train Dice: 0.4743	Val
Epoch 14/50	Train Loss: 0.6579	Train IoU: 0.3532	Train Dice: 0.4798	Val
Epoch 15/50	Train Loss: 0.6162	Train IoU: 0.3950	Train Dice: 0.5202	Val
Epoch 16/50	Train Loss: 0.6310	Train IoU: 0.3827	Train Dice: 0.5061	Val
Epoch 17/50	Train Loss: 0.6150	Train IoU: 0.3894	Train Dice: 0.5131	Val
Epoch 18/50	Train Loss: 0.6269	Train IoU: 0.3732	Train Dice: 0.4992	Val
Epoch 19/50	Train Loss: 0.5799	Train IoU: 0.4215	Train Dice: 0.5565	Val
Epoch 20/50	Train Loss: 0.6133	Train IoU: 0.3894	Train Dice: 0.5143	Val
Epoch 21/50	Train Loss: 0.6021	Train IoU: 0.4008	Train Dice: 0.5297	Val
Epoch 22/50	Train Loss: 0.6014	Train IoU: 0.3930	Train Dice: 0.5243	Val
Epoch 23/50	Train Loss: 0.6079	Train IoU: 0.3966	Train Dice: 0.5297	Val
Epoch 24/50	Train Loss: 0.6156	Train IoU: 0.3858	Train Dice: 0.5114	Val
Epoch 25/50	Train Loss: 0.6006	Train IoU: 0.3921	Train Dice: 0.5204	Val
Epoch 26/50	Train Loss: 0.5963	Train IoU: 0.4033	Train Dice: 0.5347	Val
Epoch 27/50	Train Loss: 0.5617	Train IoU: 0.4176	Train Dice: 0.5494	Val
Epoch 28/50	Train Loss: 0.5923	Train IoU: 0.4061	Train Dice: 0.5370	Val
Epoch 29/50	Train Loss: 0.5733	Train IoU: 0.4189	Train Dice: 0.5553	Val
Epoch 30/50	Train Loss: 0.5664	Train IoU: 0.4178	Train Dice: 0.5495	Val
Epoch 31/50	Train Loss: 0.5610	Train IoU: 0.4316	Train Dice: 0.5652	Val
Epoch 32/50	Train Loss: 0.5571	Train IoU: 0.4263	Train Dice: 0.5578	Val
Epoch 33/50	Train Loss: 0.5361	Train IoU: 0.4480	Train Dice: 0.5826	Val
Epoch 34/50	Train Loss: 0.5463	Train IoU: 0.4381	Train Dice: 0.5735	Val
Epoch 35/50	Train Loss: 0.5763	Train IoU: 0.4114	Train Dice: 0.5471	Val
Epoch 36/50	Train Loss: 0.5447	Train IoU: 0.4395	Train Dice: 0.5756	Val
Epoch 37/50	Train Loss: 0.5640	Train IoU: 0.4264	Train Dice: 0.5551	Val
Epoch 38/50	Train Loss: 0.5555	Train IoU: 0.4318	Train Dice: 0.5648	Val
Epoch 39/50	Train Loss: 0.5479	Train IoU: 0.4412	Train Dice: 0.5705	Val
Epoch 40/50	Train Loss: 0.5190	Train IoU: 0.4622	Train Dice: 0.5971	Val
Epoch 41/50	Train Loss: 0.5261	Train IoU: 0.4558	Train Dice: 0.5941	Val
Epoch 42/50	Train Loss: 0.5436	Train IoU: 0.4404	Train Dice: 0.5700	Val
Epoch 43/50	Train Loss: 0.5509	Train IoU: 0.4379	Train Dice: 0.5637	Val
Epoch 44/50	Train Loss: 0.5241	Train IoU: 0.4449	Train Dice: 0.5723	Val
Epoch 45/50	Train Loss: 0.5471	Train IoU: 0.4325	Train Dice: 0.5680	Val
Epoch 46/50	Train Loss: 0.5464	Train IoU: 0.4299	Train Dice: 0.5604	Val
Epoch 47/50	Train Loss: 0.5282	Train IoU: 0.4558	Train Dice: 0.5927	Val
Epoch 48/50	Train Loss: 0.5610	Train IoU: 0.4279	Train Dice: 0.5565	Val
Epoch 49/50	Train Loss: 0.5829	Train IoU: 0.4090	Train Dice: 0.5422	Val
Epoch 50/50	Train Loss: 0.5235	Train IoU: 0.4631	Train Dice: 0.6003	Val

✓ Done. Best val loss: 0.4353

Part 10: Plot Training Curves

Visualise val IoU over epochs to check convergence and noise level. A good v3 run should show val IoU climbing smoothly to >0.60 and not swinging ± 0.05 between consecutive epochs.

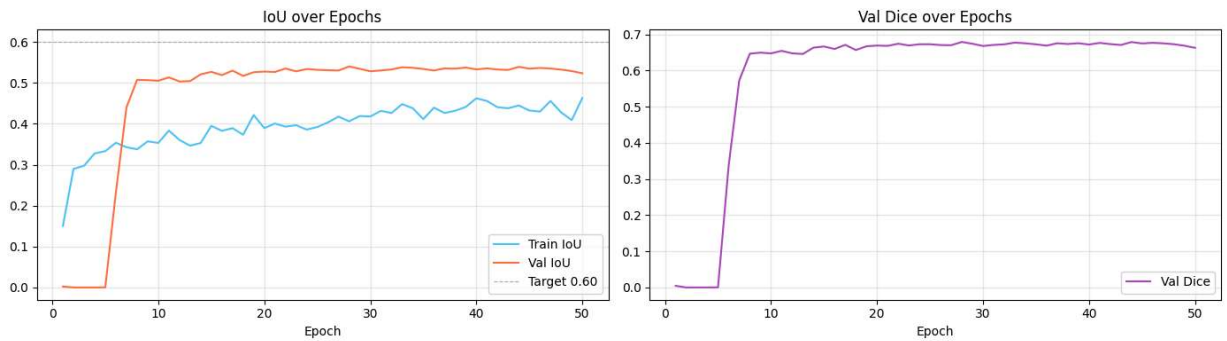
```
epochs_list = [r["epoch"] for r in history]
val_iou      = [r["val_iou"] for r in history]
train_iou    = [r["train_iou"] for r in history]
val_dice     = [r["val_dice"] for r in history]

fig, axes = plt.subplots(1, 2, figsize=(14, 4))

axes[0].plot(epochs_list, train_iou, label="Train IoU", color="#4fc3f7")
axes[0].plot(epochs_list, val_iou, label="Val IoU", color="#ff7043")
axes[0].axhline(0.60, color="#aaa", linestyle="--", linewidth=0.8, label="Target")
axes[0].set_title("IoU over Epochs")
axes[0].set_xlabel("Epoch")
axes[0].legend()
axes[0].grid(alpha=0.3)

axes[1].plot(epochs_list, val_dice, label="Val Dice", color="#ab47bc")
axes[1].set_title("Val Dice over Epochs")
axes[1].set_xlabel("Epoch")
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```



✓ Part 11: Final Test Set Evaluation

Load the best checkpoint and evaluate on the **held-out 10% test set**. These are the only numbers you report in your paper/presentation. They were never seen during training or scheduler tuning.

```
model.load_state_dict(torch.load("/content/flood_unet_resnet34_best.pth", map_1
model.eval()

test_iou = test_dice = 0.0
with torch.no_grad():
    for images, masks in test_loader:
        images, masks = images.to(device), masks.to(device)
        outputs = model(images)
        test_iou += iou_score(outputs, masks).item()
        test_dice += dice_score(outputs, masks).item()

n_test = len(test_loader)
print("=" * 55)
print(f"  FINAL TEST SET RESULTS  ({len(test_aug)} held-out samples)")
print(f"  Test IoU   : {test_iou / n_test:.4f}")
print(f"  Test Dice  : {test_dice / n_test:.4f}")
print("=" * 55)
print("  Record these numbers in PROJECT_NOTES.md")
```

```
=====
FINAL TEST SET RESULTS (44 held-out samples)
Test IoU : 0.4819
Test Dice : 0.6243
=====
Record these numbers in PROJECT_NOTES.md
```

✓ Part 12: Visualize Predictions

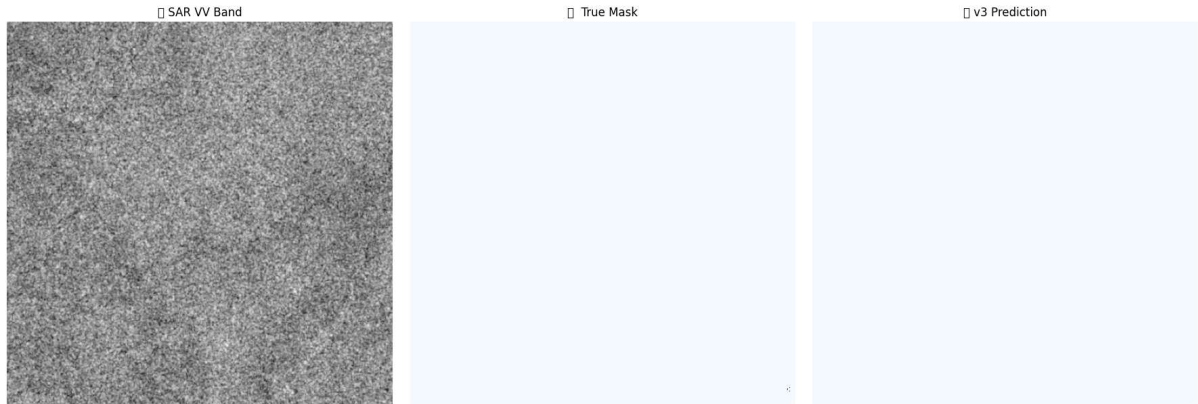
```
def visualize_prediction(model, dataset, index=0):
    model.eval()
    image, true_mask = dataset[index]
    with torch.no_grad():
        pred = torch.sigmoid(model(image.unsqueeze(0).to(device)))
        pred_mask = (pred > 0.5).float().cpu().numpy().squeeze()

    fig, axes = plt.subplots(1, 3, figsize=(18, 6))
    axes[0].imshow(image[0].numpy(), cmap="gray"); axes[0].set_title("Original Image")
    axes[1].imshow(true_mask.numpy().squeeze(), cmap="Blues"); axes[1].set_title("True Mask")
    axes[2].imshow(pred_mask, cmap="Blues"); axes[2].set_title("Predicted Mask")
    plt.tight_layout(); plt.show()

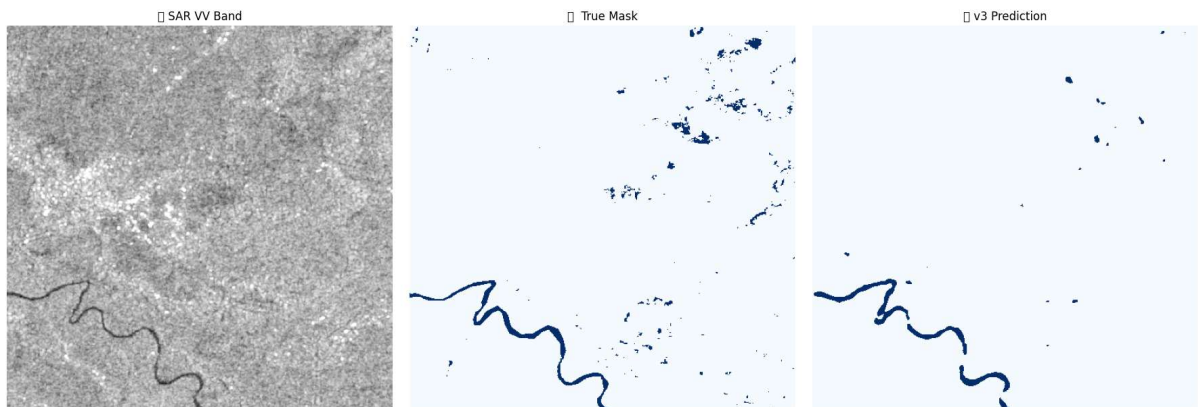
# Try a few different val samples
for i in [5, 10, 15, 20]:
    print(f"--- Sample {i} ---")
    visualize_prediction(model, val_aug, index=i)
```


--- Sample 5 ---

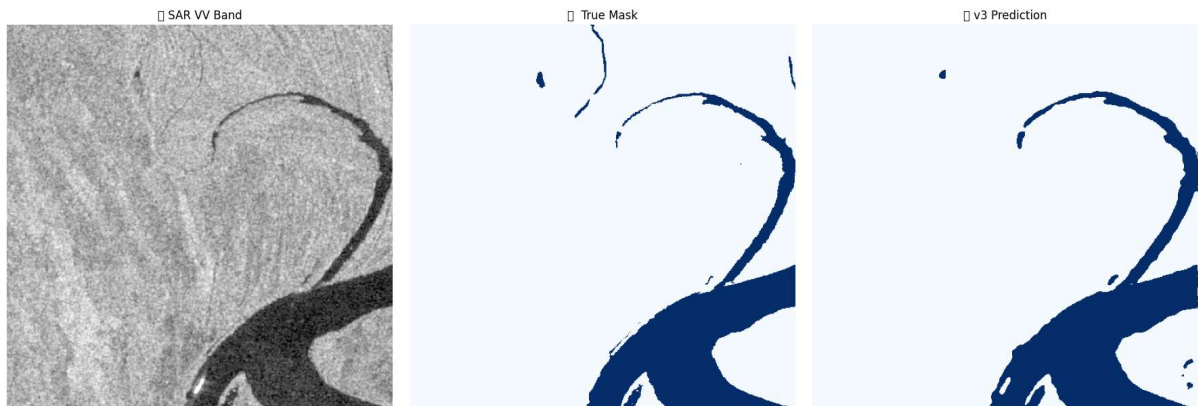
```
/tmp/ipykernel_563/3772653808.py:12: UserWarning: Glyph 128225 (\N{SATELLITE A
plt.tight_layout(); plt.show()
/tmp/ipykernel_563/3772653808.py:12: UserWarning: Glyph 128506 (\N{WORLD MAP})
plt.tight_layout(); plt.show()
/tmp/ipykernel_563/3772653808.py:12: UserWarning: Glyph 129302 (\N{ROBOT FACE})
plt.tight_layout(); plt.show()
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWa
fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWa
fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWa
fig.canvas.print_figure(bytes_io, **kw)
```



--- Sample 10 ---



--- Sample 15 ---



--- Sample 20 ---

